

DJANGO-MYSQL CONNECTION

Overview

This document explains how to connect a Django web application to a MySQL database. It covers the full process from scratch — installing MySQL 8.0 on Windows, creating a database, setting up a Django project, defining models, running migrations, and verifying the data in MySQL.

Step 1: Installing MySQL 8.0 on Windows

MySQL is a popular open-source relational database. We install it using the official MySQL Installer for Windows which bundles MySQL Server, MySQL Workbench, MySQL Shell, and other tools.

1.1 Search and Open MySQL Workbench / Installer

After downloading the MySQL Community Installer (.msi), search for it from the Windows Start menu. You will see MySQL Workbench, MySQL Shell, MySQL 8.0 Command Line Client, and the MySQL Installer listed.

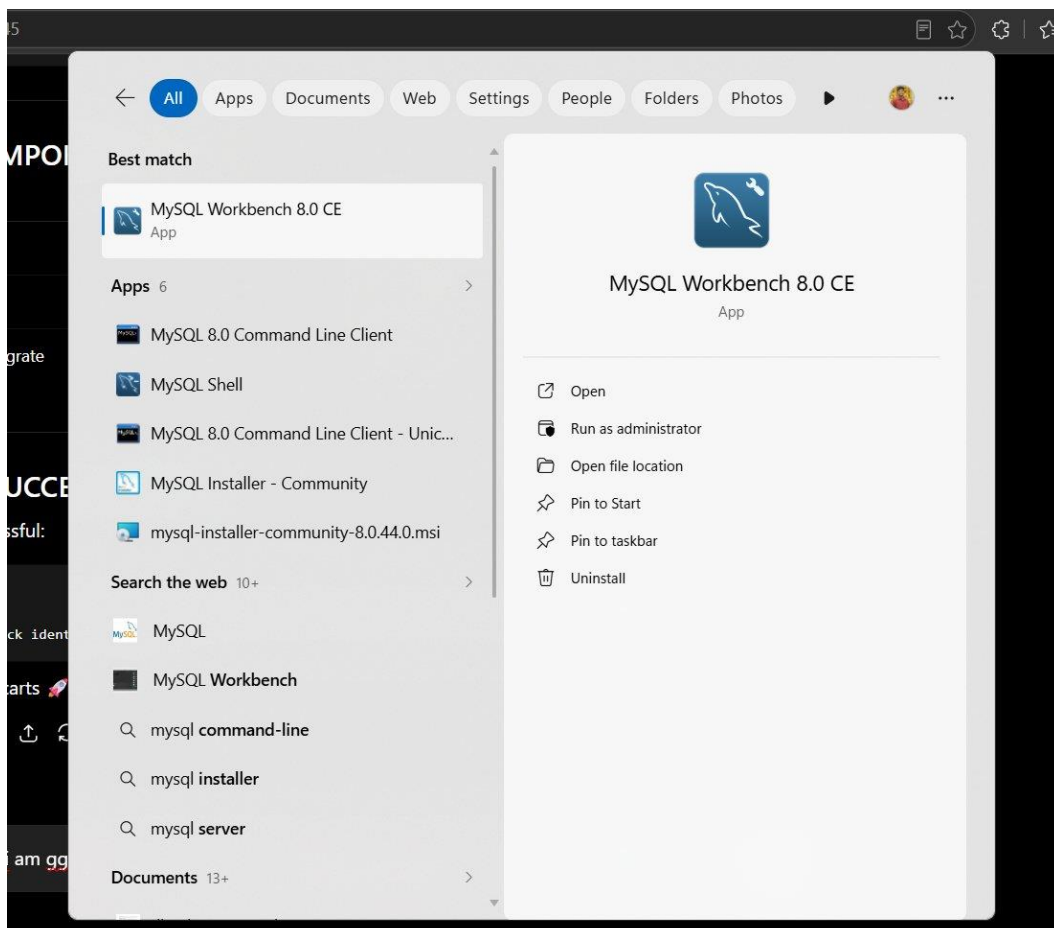


Figure 1: MySQL apps visible in Windows Start Menu after installation

1.2 Download MySQL Server

The MySQL Installer will download MySQL Server 8.0 (x64 architecture). The download screen shows progress. Wait for it to reach 100% before proceeding.

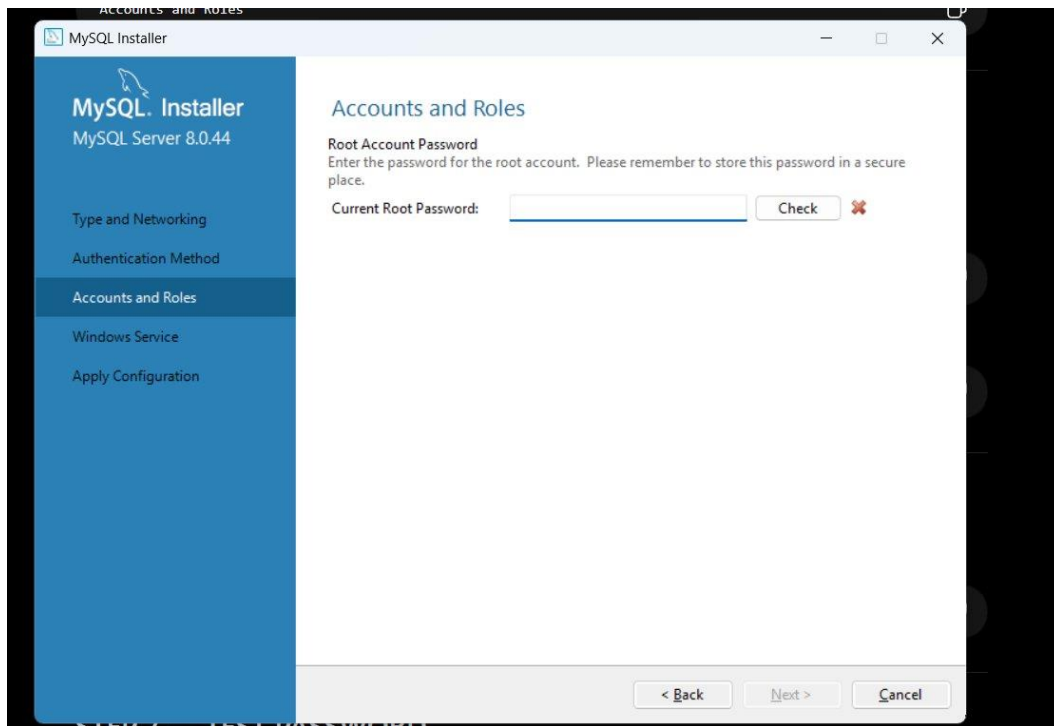


Figure 2: MySQL Installer downloading MySQL Server 8.0

1.3 Configure Type and Networking

Once downloaded, the installer walks through configuration. On the "Type and Networking" screen, set the Config Type to "Development Computer" and keep TCP/IP enabled on port 3306. Enable the Windows Firewall port option.

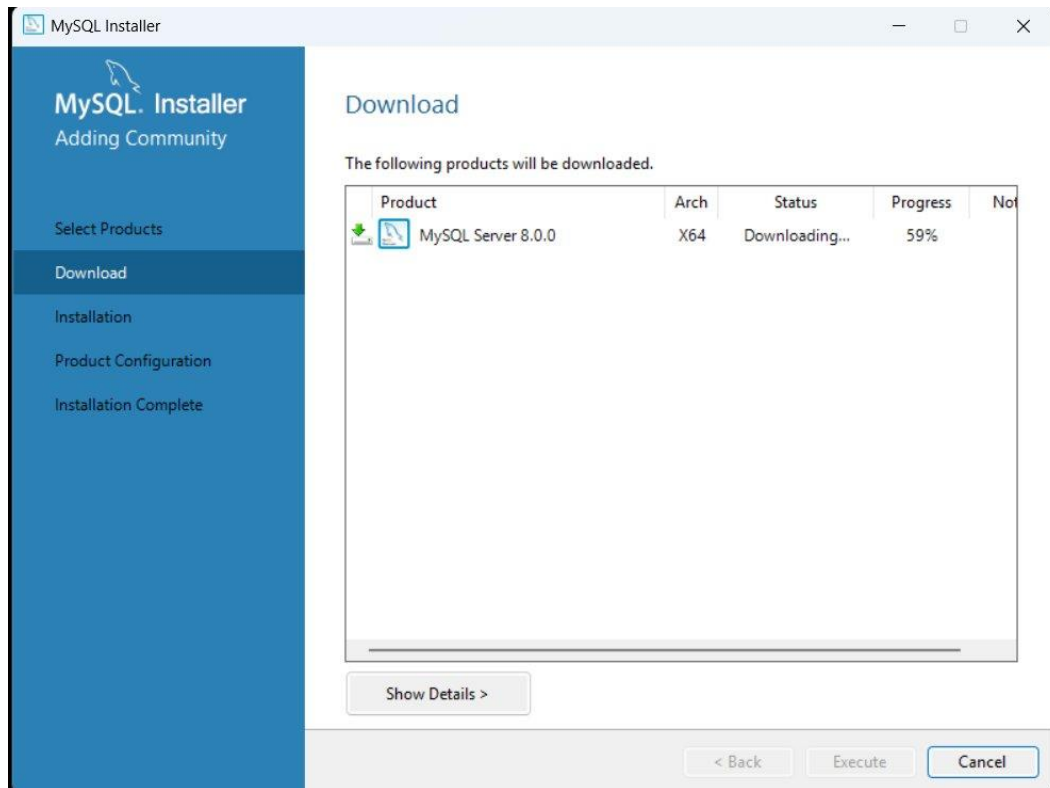


Figure 3: Type and Networking — keep default port 3306

1.4 Set the Root Password

In the "Accounts and Roles" screen, set the MySQL root account password. Remember this password — you will need it in the Django settings.py later. The installer shows password strength feedback.

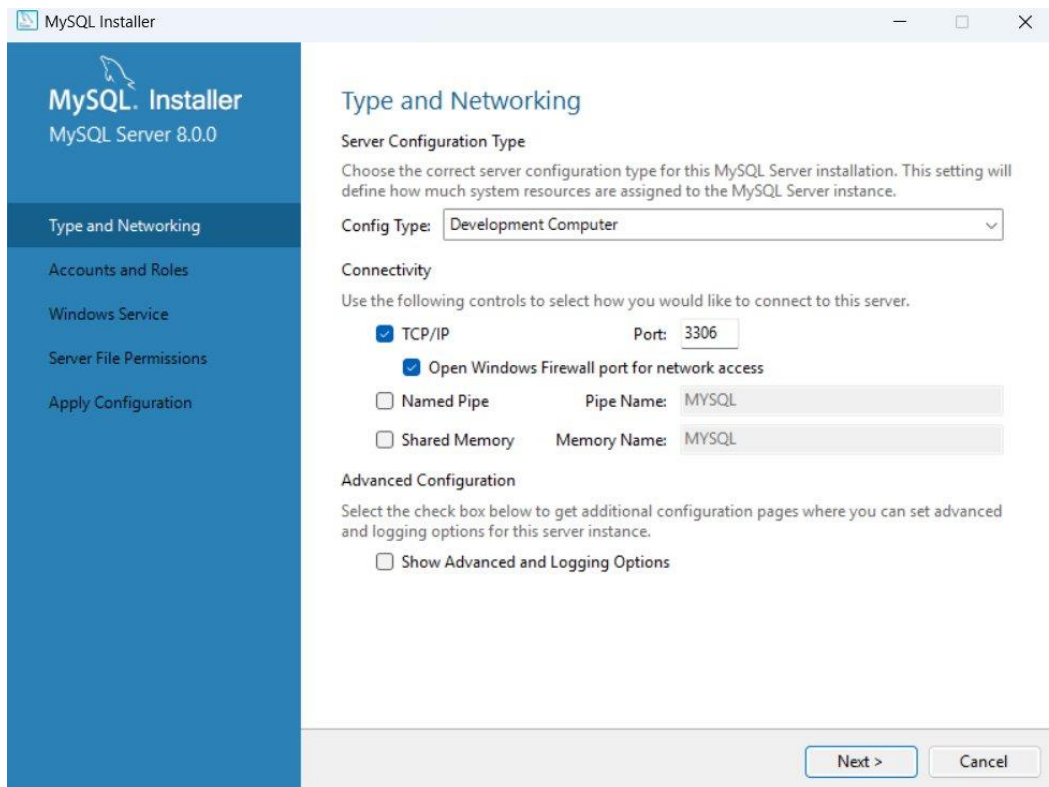


Figure 4: Setting MySQL root password during installation

1.5 Apply Configuration (In Progress)

The installer runs through all configuration steps: writing config file, updating firewall rules, adjusting Windows service, initializing the database, setting file permissions, starting the server, applying security settings, and updating the Start menu.

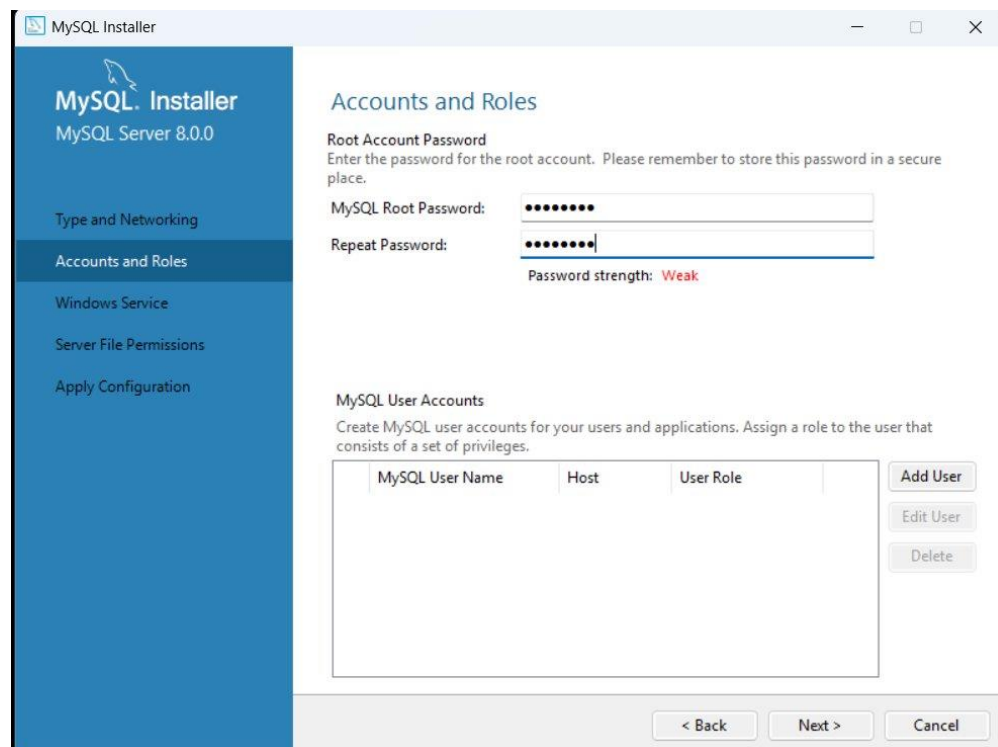


Figure 5: Apply Configuration running (database initializing)

1.6 Configuration Completed Successfully

All steps complete with green checkmarks. Click Finish to proceed.

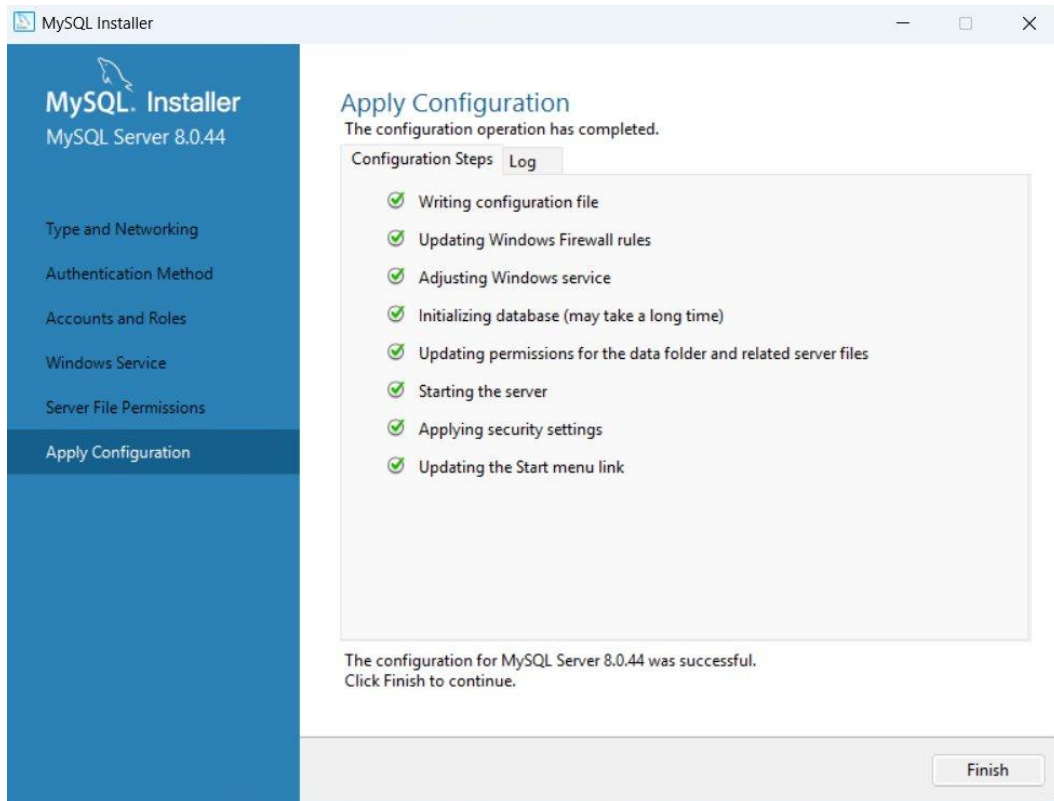


Figure 6: MySQL Server 8.0.44 configured successfully

1.7 Installation Complete

The MySQL Installer shows "Installation Complete". All products including MySQL Server 8.0.44, MySQL Workbench, MySQL Shell, MySQL Router, Documentation, and Samples are now installed.

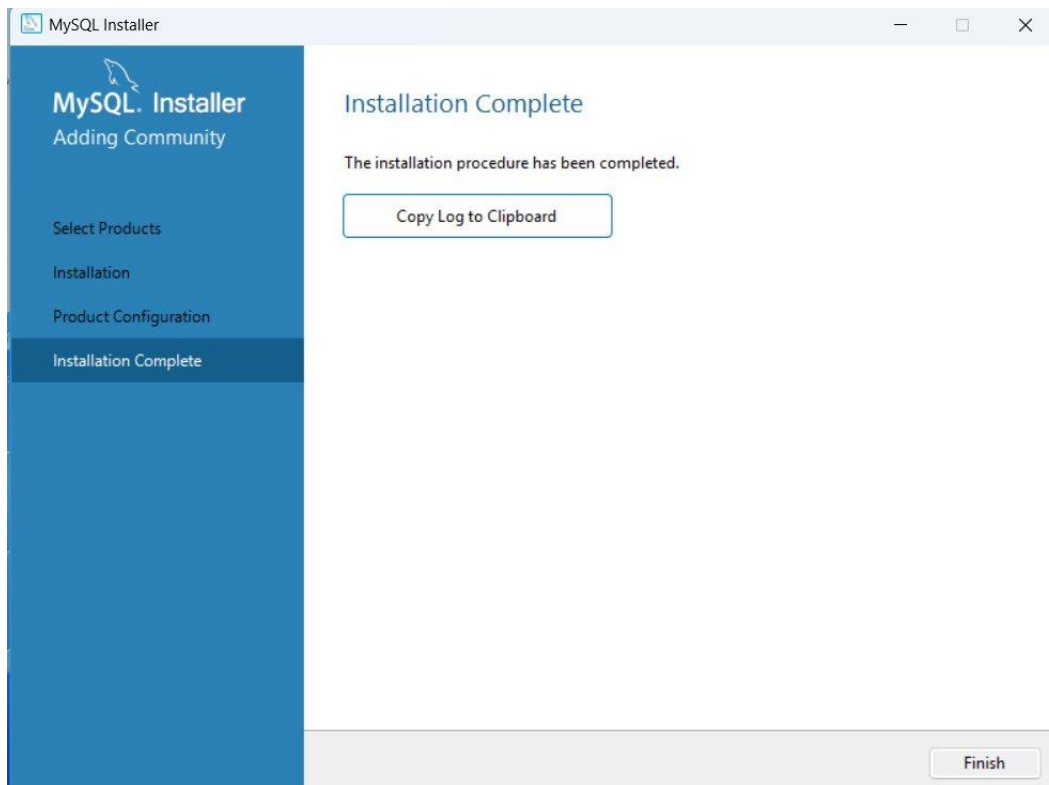


Figure 7: Installation Complete screen

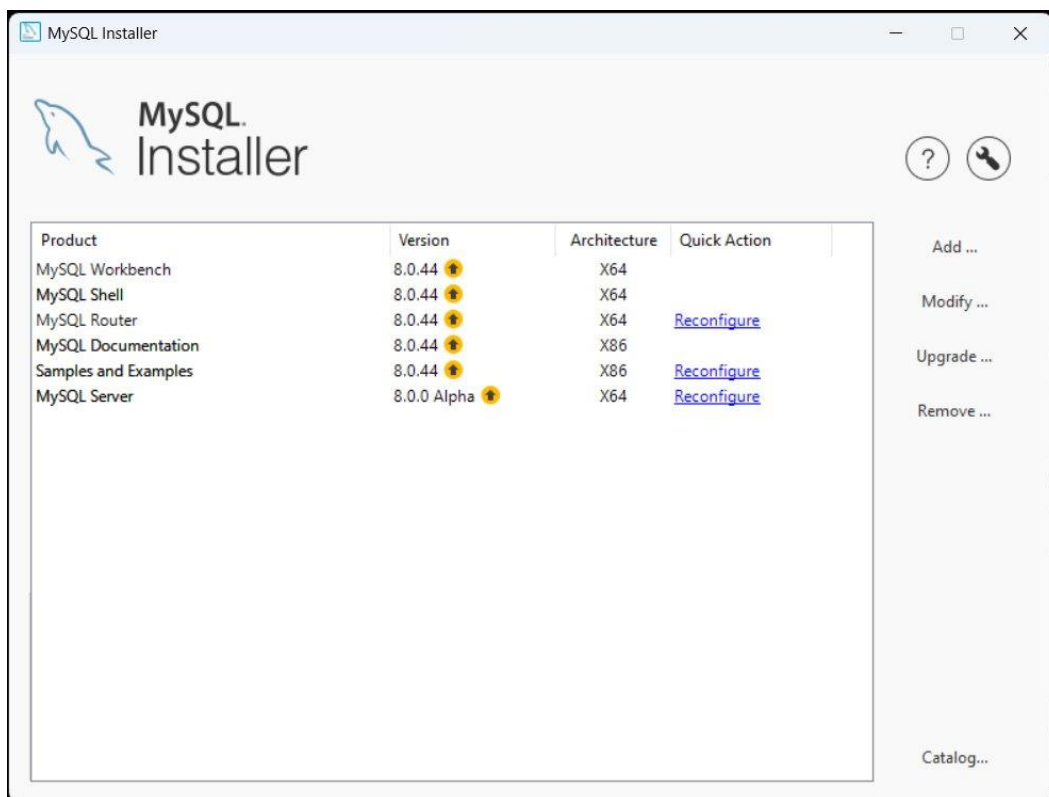


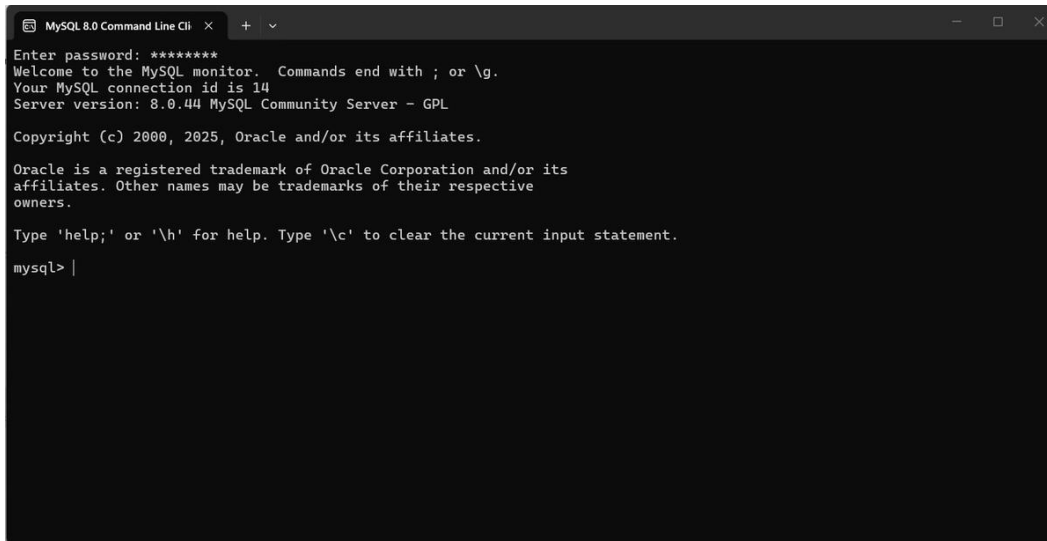
Figure 8: MySQL Installer product list showing all installed components

Step 2: Using MySQL Command Line

With MySQL installed, we now create a database for our Django project using the MySQL 8.0 Command Line Client.

2.1 Login to MySQL CLI

Open "MySQL 8.0 Command Line Client" from the Start menu. Enter your root password. You will see the MySQL monitor welcome message with your connection ID and server version.



```
MySQL 8.0 Command Line Cli  x  +  v
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 14
Server version: 8.0.44 MySQL Community Server - GPL

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

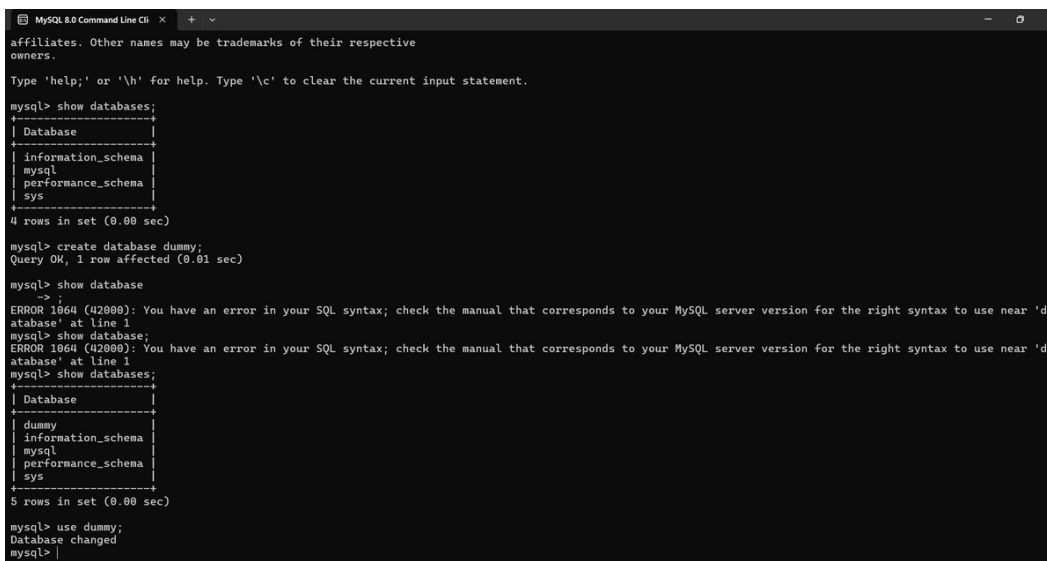
mysql> |
```

Figure 9: Successfully logged into MySQL 8.0.44 Community Server

2.2 View Databases and Create a New One

Run SHOW DATABASES to see the existing system databases (information_schema, mysql, performance_schema, sys). Then create a new database called "dummy" for this project.

```
mysql> show databases;
mysql> create database dummy;
mysql> use dummy;
```



```
MySQL 8.0 Command Line Cli  x  +  v
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> show databases;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| sys |
+-----+
4 rows in set (0.00 sec)

mysql> create database dummy;
Query OK, 1 row affected (0.01 sec)

mysql> show database
->
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'd
atabase' at line 1
mysql> show database;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'd
atabase' at line 1
mysql> show databases;
+-----+
| Database |
+-----+
| dummy |
| information_schema |
| mysql |
| performance_schema |
| sys |
+-----+
5 rows in set (0.00 sec)

mysql> use dummy;
Database changed
mysql> |
```

Figure 10: Creating the "dummy" database and switching to it

Step 3: Setting Up the Django Project

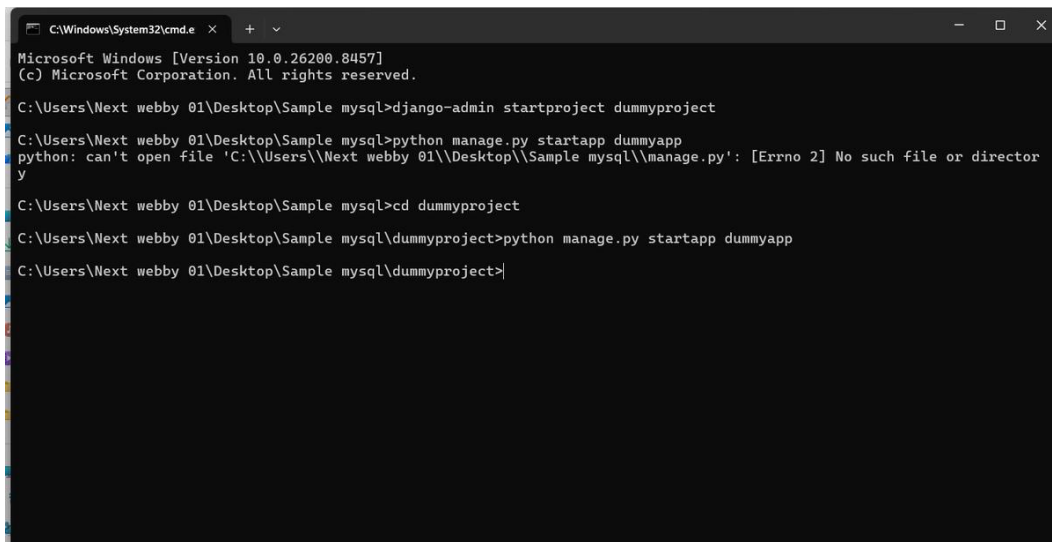
Now we create a Django project and app. Django is a Python web framework. Make sure Python and Django are already installed. Install Django using pip if needed:

```
pip install django
pip install mysqlclient
```

3.1 Create the Django Project and App

Navigate to your desired folder in the Command Prompt, then run the following commands:

```
django-admin startproject dummyproject
cd dummyproject
python manage.py startapp dummyapp
```



```
C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Next webby 01\Desktop\Sample mysql>django-admin startproject dummyproject

C:\Users\Next webby 01\Desktop\Sample mysql>python manage.py startapp dummyapp
python: can't open file 'C:\\Users\\Next webby 01\\Desktop\\Sample mysql\\manage.py': [Errno 2] No such file or directory

C:\Users\Next webby 01\Desktop\Sample mysql>cd dummyproject

C:\Users\Next webby 01\Desktop\Sample mysql\dummyproject>python manage.py startapp dummyapp

C:\Users\Next webby 01\Desktop\Sample mysql\dummyproject>|
```

Figure 11: Creating the Django project and app using command prompt

3.2 Define the Model in models.py

Open dummyapp/models.py in VS Code and create a Student model with two fields: name (CharField) and age (IntegerField). Django will automatically add an "id" primary key field.

```
from django.db import models

class Student(models.Model):
    name = models.CharField(max_length=100)
    age = models.IntegerField()

    def __str__(self):
        return self.name
```

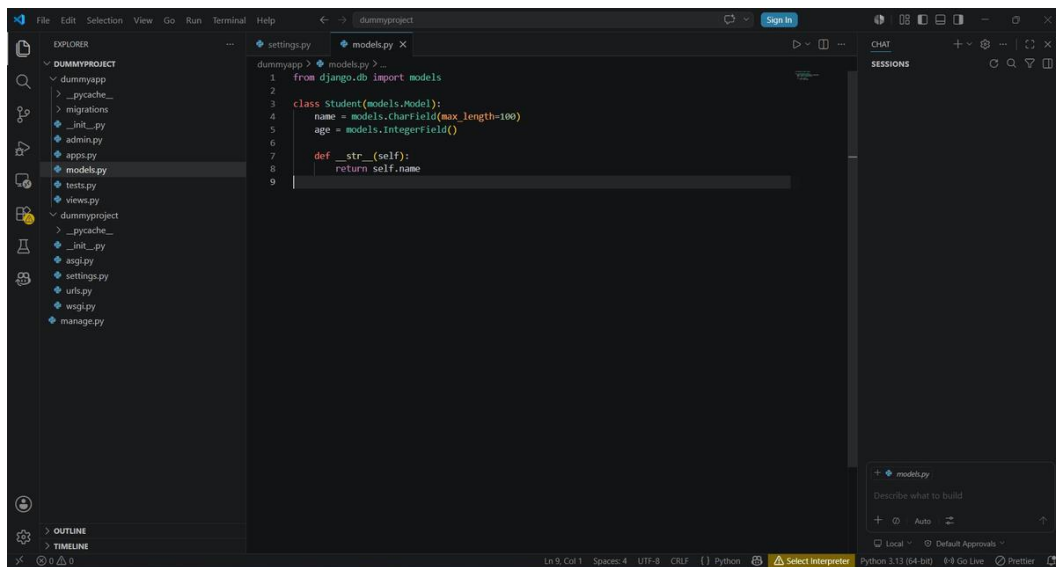


Figure 12: Student model defined in dummyapp/models.py in VS Code

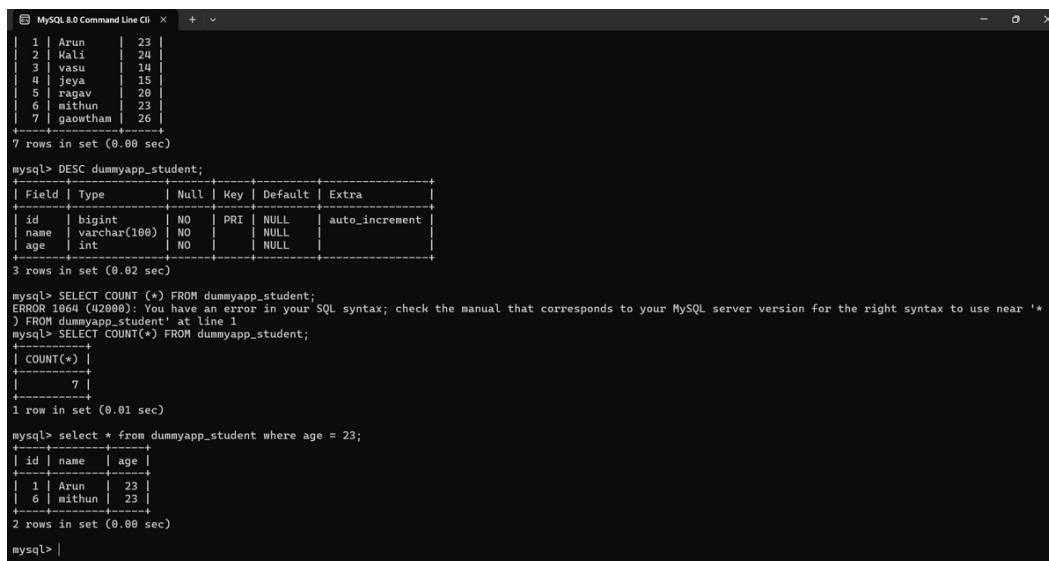
Step 4: Configuring Django to Use MySQL

By default, Django uses SQLite. We need to change the DATABASES setting in settings.py to use MySQL instead.

4.1 Update settings.py — DATABASES section

Open dummyproject/settings.py and replace the default SQLite DATABASES block with the MySQL configuration:

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.mysql',
        'NAME': 'dummy',
        'USER': 'root',
        'PASSWORD': 'root@123',
        'HOST': 'localhost',
        'PORT': '3306',
    }
}
```



```
mysql> SELECT * FROM dummyapp_student;
+----+-----+-----+
| 1 | Arun   | 23 |
| 2 | Kati   | 24 |
| 3 | vasu   | 14 |
| 4 | jeya   | 15 |
| 5 | ragav  | 20 |
| 6 | mithun | 23 |
| 7 | gaoutham | 25 |
+----+-----+-----+
7 rows in set (0.00 sec)

mysql> DESC dummyapp_student;
+----+-----+-----+
| Field | Type      | Null | Key | Default | Extra      |
+----+-----+-----+
| id     | bigint    | NO   | PRI | NULL    | auto_increment |
| name   | varchar(100) | NO   |     | NULL    |              |
| age    | int       | NO   |     | NULL    |              |
+----+-----+-----+
3 rows in set (0.02 sec)

mysql> SELECT COUNT(*) FROM dummyapp_student;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near '*'
) FROM dummyapp_student' at line 1
mysql> SELECT COUNT(*) FROM dummyapp_student;
+----+
| COUNT(*) |
+----+
| 7        |
+----+
1 row in set (0.01 sec)

mysql> select * from dummyapp_student where age = 23;
+----+-----+-----+
| id | name  | age |
+----+-----+-----+
| 1  | Arun  | 23  |
| 6  | mithun | 23  |
+----+-----+-----+
2 rows in set (0.00 sec)

mysql> |
```

Figure 13: MySQL database configuration in dummyproject/settings.py

4.2 Register the App in INSTALLED_APPS

In settings.py, also add "dummyapp" to the INSTALLED_APPS list so Django recognizes the app and includes its models in migrations.

```
INSTALLED_APPS = [
    ...
    'dummyapp',
]
```

Step 5: Running Migrations

Django migrations create the actual database tables from your model definitions. There are two commands involved:

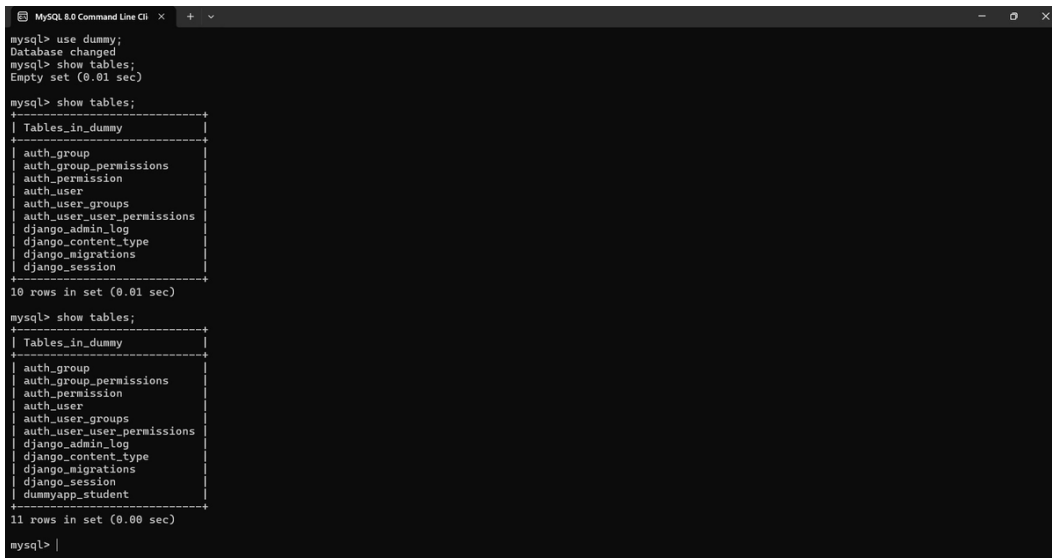
- `makemigrations` — creates migration files from your models
- `migrate` — applies those migration files to the MySQL database

5.1 Run makemigrations and migrate

```
python manage.py makemigrations
python manage.py migrate
```

5.2 Verify Tables in MySQL

After running migrate, switch to the MySQL CLI and run `SHOW TABLES` in the dummy database. You will see Django's built-in tables (`auth_*`, `django_*`) and — importantly — the `dummyapp_student` table created from our model.



```
mysql> use dummy;
Database changed
mysql> show tables;
Empty set (0.01 sec)

mysql> show tables;
+-----+
| Tables_in_dummy |
+-----+
| auth_group       |
| auth_group_permissions |
| auth_permission  |
| auth_user        |
| auth_user_groups |
| auth_user_user_permissions |
| django_admin_log |
| django_content_type |
| django_migrations |
| django_session   |
+-----+
10 rows in set (0.01 sec)

mysql> show tables;
+-----+
| Tables_in_dummy |
+-----+
| auth_group       |
| auth_group_permissions |
| auth_permission  |
| auth_user        |
| auth_user_groups |
| auth_user_user_permissions |
| django_admin_log |
| django_content_type |
| django_migrations |
| django_session   |
| dummyapp_student |
+-----+
11 rows in set (0.00 sec)

mysql> |
```

Figure 14: After migrate — 11 tables created including `dummyapp_student`

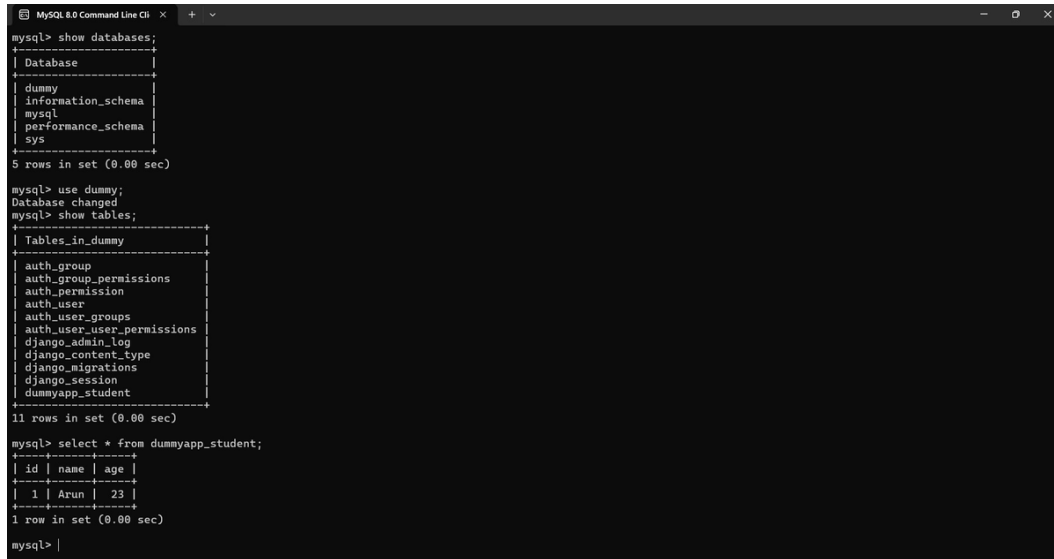
Step 6: Inserting and Querying Data

Now the table exists in MySQL. We can insert data either through Django admin/shell or directly in MySQL CLI. Here we use MySQL CLI for demonstration.

6.1 First Record — via Django Admin/Shell

The first student record (Arun, age 23) was added via the Django interface. Verifying it in MySQL:

```
mysql> select * from dummyapp_student;
```



```
mysql> show databases;
+-----+
| Database |
+-----+
| dummy    |
| information_schema |
| mysql    |
| performance_schema |
| sys      |
+-----+
5 rows in set (0.00 sec)

mysql> use dummy;
Database changed
mysql> show tables;
+-----+
| Tables_in_dummy |
+-----+
| auth_group       |
| auth_group_permissions |
| auth_permission  |
| auth_user        |
| auth_user_groups |
| auth_user_user_permissions |
| django_admin_log |
| django_content_type |
| django_migrations |
| django_session   |
| dummyapp_student |
+-----+
11 rows in set (0.00 sec)

mysql> select * from dummyapp_student;
+----+-----+-----+
| id | name | age |
+----+-----+-----+
| 1  | Arun | 23  |
+----+-----+-----+
1 row in set (0.00 sec)

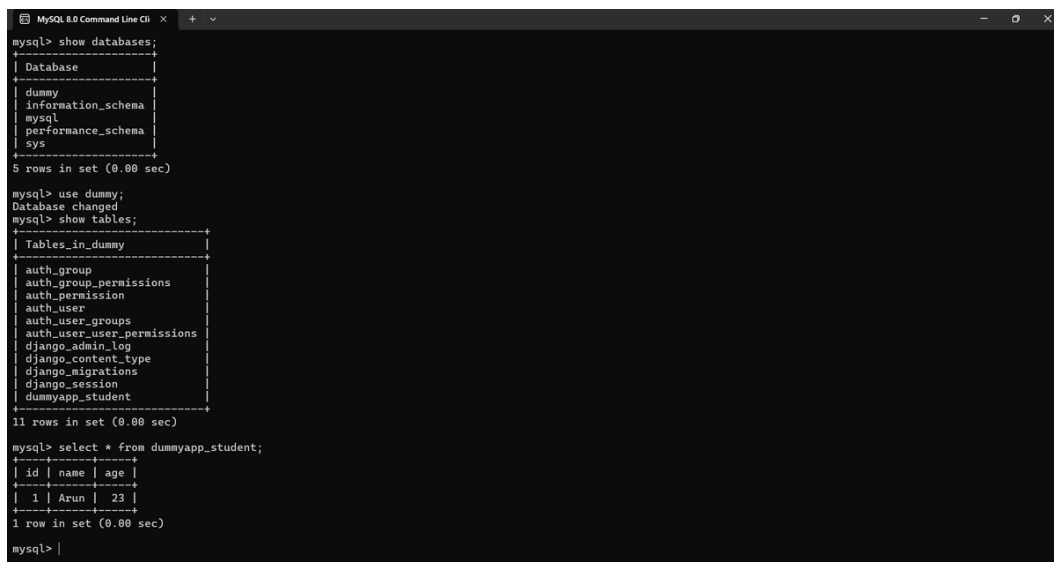
mysql> |
```

Figure 15: First record "Arun" visible in dummyapp_student table

6.2 Inserting Multiple Records via MySQL CLI

Insert several more students directly using SQL:

```
mysql> insert into dummyapp_student(name, age) values
('vasu',14), ('jeva',15), ('ragav',20), ('mithun',23), ('gaowtham',26);
```



```
mysql> show databases;
+-----+
| Database |
+-----+
| dummy    |
| information_schema |
| mysql    |
| performance_schema |
| sys      |
+-----+
5 rows in set (0.00 sec)

mysql> use dummy;
Database changed
mysql> show tables;
+-----+
| Tables_in_dummy |
+-----+
| auth_group       |
| auth_group_permissions |
| auth_permission  |
| auth_user        |
| auth_user_groups |
| auth_user_user_permissions |
| django_admin_log |
| django_content_type |
| django_migrations |
| django_session   |
| dummyapp_student |
+-----+
11 rows in set (0.00 sec)

mysql> select * from dummyapp_student;
+----+-----+-----+
| id | name | age |
+----+-----+-----+
| 1  | Arun | 23  |
+----+-----+-----+
1 row in set (0.00 sec)

mysql> |
```

Figure 16: Inserting 5 records at once — table now has 7 students

6.3 Describe the Table Structure

Use DESC to see the exact column types Django created. The id column is a bigint with auto_increment (primary key), name is varchar(100), and age is int.

```
mysql> DESC dummyapp_student;
```

6.4 Count Records

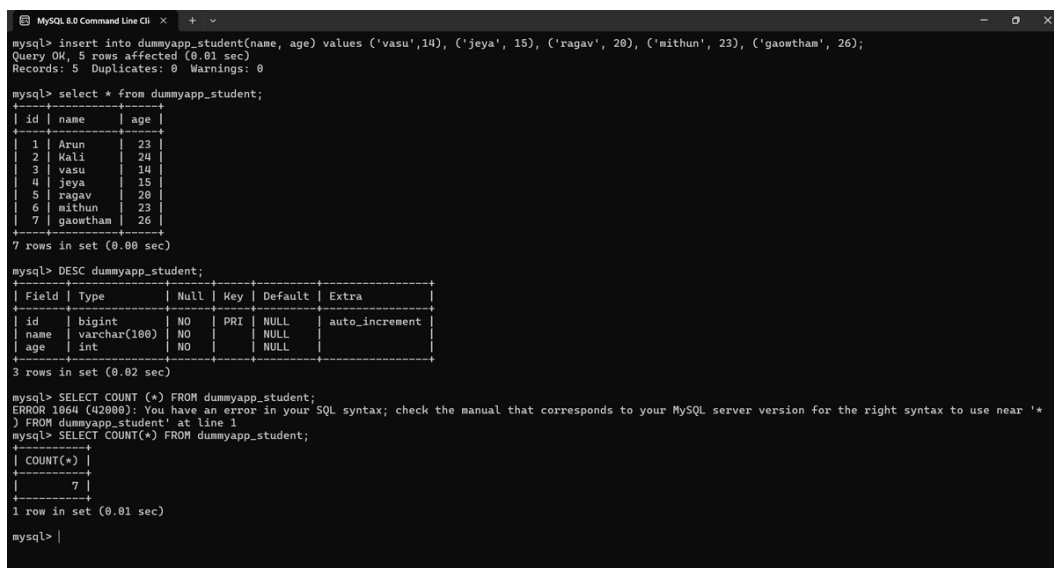
```
mysql> SELECT COUNT(*) FROM dummyapp_student;
```

Result: 7 rows — confirming all records are stored correctly.

6.5 Filter with WHERE Clause

Query students with a specific age using the WHERE clause:

```
mysql> select * from dummyapp_student where age = 23;
```



```
mysql> insert into dummyapp_student(name, age) values ('vasu',14), ('jeya', 15), ('ragav', 20), ('mithun', 23), ('gaowtham', 26);
Query OK, 5 rows affected (0.01 sec)
Records: 5 Duplicates: 0 Warnings: 0

mysql> select * from dummyapp_student;
+----+-----+-----+
| id | name  | age |
+----+-----+-----+
| 1  | Arun  | 23  |
| 2  | Kali  | 24  |
| 3  | vasu  | 14  |
| 4  | jeya  | 15  |
| 5  | ragav | 20  |
| 6  | mithun | 23  |
| 7  | gaowtham | 26  |
+----+-----+-----+
7 rows in set (0.00 sec)

mysql> DESC dummyapp_student;
+----+-----+-----+-----+-----+-----+
| Field | Type   | Null | Key | Default | Extra |
+----+-----+-----+-----+-----+-----+
| id    | bigint | NO   | PRI | NULL    | auto_increment |
| name  | varchar(100) | NO   |     | NULL    |               |
| age   | int    | NO   |     | NULL    |               |
+----+-----+-----+-----+-----+-----+
3 rows in set (0.02 sec)

mysql> SELECT COUNT(*) FROM dummyapp_student;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near '*'
) FROM dummyapp_student' at line 1
mysql> SELECT COUNT(*) FROM dummyapp_student;
+-----+
| COUNT(*) |
+-----+
| 7        |
+-----+
1 row in set (0.01 sec)

mysql> |
```

Figure 17: WHERE clause filtering — Arun and Mithun both have age 23

Overall Commands:

MySQL + Django Commands Summary

Category	Command	Purpose
Show Databases	SHOW DATABASES;	Shows all databases
Create Database	CREATE DATABASE dummy;	Creates new database
Use Database	USE dummy;	Selects database
Current Database	SELECT DATABASE();	Shows current database
Delete Database	DROP DATABASE dummy;	Deletes database

TABLE COMMANDS

Command Type	Command	Purpose
Show Tables	SHOW TABLES;	Shows all tables
Create Table	CREATE TABLE student(id INT, name VARCHAR(100), age INT);	Creates table
Describe Table	DESC student;	Shows table structure
Rename Table	RENAME TABLE student TO students;	Renames table
Delete Table	DROP TABLE student;	Deletes table

INSERT COMMANDS

Command Type	Command	Purpose
Insert Single Row	INSERT INTO student VALUES(1,'Arun',23);	Inserts one record
Insert Specific Columns	INSERT INTO student(name,age) VALUES('Ravi',25);	Inserts selected columns
Insert Multiple Rows	INSERT INTO student(name,age) VALUES ('Kumar',30),('Suresh',28);	Inserts multiple records

SELECT COMMANDS

Command Type	Command	Purpose
View All Data	SELECT * FROM student;	Displays all rows
View Specific Columns	SELECT name,age FROM student;	Displays selected columns
WHERE Condition	SELECT * FROM student WHERE age=23;	Filters data
AND Condition	SELECT * FROM student WHERE age=23 AND name='Arun';	Multiple conditions
OR Condition	SELECT * FROM student WHERE age=23 OR age=24;	Either condition

UPDATE & DELETE COMMANDS

Command Type	Command	Purpose
Update Data	UPDATE student SET age=26 WHERE id=1;	Updates record
Delete Row	DELETE FROM student WHERE id=1;	Deletes one row
Delete All Rows	TRUNCATE TABLE student;	Clears table data

ALTER TABLE COMMANDS

Command Type	Command	Purpose
Add Column	ALTER TABLE student ADD email VARCHAR(100);	Adds column
Remove Column	ALTER TABLE student DROP COLUMN email;	Deletes column
Change Column Name	ALTER TABLE student CHANGE name full_name VARCHAR(100);	Renames column

SORTING & LIMIT COMMANDS

Command Type	Command	Purpose
Ascending Order	SELECT * FROM student ORDER BY age ASC;	Sorts ascending
Descending Order	SELECT * FROM student ORDER BY age DESC;	Sorts descending
Limit Rows	SELECT * FROM student LIMIT 2;	Shows limited rows

SEARCH COMMANDS

Command Type	Command	Purpose
Starts With	SELECT * FROM student WHERE name LIKE 'A%';	Names starting with A
Ends With	SELECT * FROM student WHERE name LIKE "%n";	Names ending with n
Contains	SELECT * FROM student WHERE name LIKE "%ru%";	Contains text

COUNT & FUNCTIONS

Command Type	Command	Purpose
Count Rows	SELECT COUNT(*) FROM student;	Counts records
Maximum Value	SELECT MAX(age) FROM student;	Highest value
Minimum Value	SELECT MIN(age) FROM student;	Lowest value
Average	SELECT AVG(age) FROM student;	Average value
Sum	SELECT SUM(age) FROM student;	Total sum

KEY CONCEPTS

Command Type	Command	Purpose
--------------	---------	---------

Primary Key	id INT PRIMARY KEY	Unique identifier
Auto Increment	id INT AUTO_INCREMENT PRIMARY KEY	Automatic ID
Foreign Key	FOREIGN KEY(student_id) REFERENCES student(id)	Connects tables

MYSQL TERMINAL COMMANDS

Command	Purpose
system cls	Clears terminal
EXIT;	Exits MySQL
QUIT;	Exits MySQL

DJANGO COMMANDS

Command	Purpose
django-admin startproject myproject	Creates Django project
python manage.py startapp students	Creates app
python manage.py makemigrations	Creates migration files
python manage.py migrate	Applies migrations
python manage.py runserver	Runs Django server
python manage.py createsuperuser	Creates admin user

DJANGO MYSQL CONFIGURATION

settings.py

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.mysql',
        'NAME': 'dummy',
        'USER': 'root',
        'PASSWORD': 'root@123',
        'HOST': 'localhost',
        'PORT': '3306',
    }
}
```